

SignDesk

A digital document signing platform: send a PDF by email, sign it in the browser, and get back a cryptographically sealed file. Prepared August 2026.

This document covers how to run and use the system, how a document actually flows through it, and the five questions set with the assignment: the tech stack, why that stack and not the alternatives, the competition and their pricing, the security case for building in-house rather than buying, and the core concepts of digital signatures.

Everything described here was built and runs. Every claim about signature levels, trusted timestamps and tamper detection is reproducible with the commands in section 10 - and is confirmed by pyHanko's command-line validator, which is not part of this codebase.

What it does

- Admin portal: upload a PDF, click to place fields per recipient, send, track status.
- Signer portal: tokenised email link, one-time passcode, consent, sign, done.
- Three signature modes: draw, upload a PNG or JPEG, or type a name in a script font.
- Auto-generated Certificate of Completion recording identity method, consent version, document hashes and the full event timeline.
- **PAdES-B-LTA** sealing: PKCS#7 signature, RFC 3161 trusted timestamp, embedded revocation data and a renewable document-timestamp chain.
- Hash-chained, append-only audit trail enforced by database triggers.
- Public verification page: upload any PDF and check whether it has been altered.

1. Running the system

Requires PHP 8.3 (with the pdo_pgsql extension), Composer, Node 20+, Python 3.12, Docker and OpenSSL. Three processes run: a React SPA, a Laravel API, and a Python sealing service.

Step 1 - infrastructure

```
docker compose up -d postgres redis minio minio-init mailpit pki-web
```

Postgres is published on 55432 and Redis on 63790, deliberately high. A native PostgreSQL install on 5432 or 5433 would otherwise answer instead, which surfaces as a confusing authentication failure rather than a connection error.

Step 2 - certificates

```
bash pki/scripts/gen-pki.sh
```

Creates a root CA, a document-signing certificate and a CRL, served by the pki-web container on port 8080. **This is not optional.** PAdES B-LT and B-LTA embed revocation data, and with the distribution point unreachable an otherwise perfect signature validates as INVALID.

Step 3 - sealing service

```
cd sign-service
python -m venv .venv
./venv/Scripts/python.exe -m pip install -r requirements.txt
python scripts/fetch_fonts.py
PKI_DIR=../pki/out ./venv/Scripts/python.exe -m uvicorn app.main:app --port 8001
```

Step 4 - API

```
cd api
composer install
```

```
php artisan migrate --seed
php artisan serve --port=8000
php artisan queue:work # second terminal - sealing runs on the queue
```

Seeded administrator: admin@signdesk.test / password. The queue worker holds the application in memory, so restart it after changing code it touches.

Step 5 - the SPA

```
cd web
npm install
npm run dev
```

Open <http://localhost:5173>. All outbound mail is captured by Mailpit at <http://localhost:8025>, so signing links and passcodes are readable without a real mailbox.

2. Using it

As the administrator

- 1 Sign in with the seeded credentials.
- 2 **New envelope**, then choose a PDF. It is validated, hashed and stored; its pages render immediately.
- 3 Fill in each recipient's name and email. Each signer gets their own colour.
- 4 Pick a field type - signature, initial, date, text or checkbox - then click the page to drop it. Fields belong to the highlighted signer. Hover a placed field to remove it.
- 5 Add a subject and an optional note, set how long the link stays valid, then **Send for signature**.
- 6 The envelope page then shows live status, the cryptographic seal once it exists, and the full audit trail with its chain state.

As the signer

- 1 Open the emailed link. The email address is shown masked.
- 2 Request a code and enter it. Until this passes, the document itself is not served.
- 3 Read the electronic-records disclosure and agree.
- 4 Adopt a signature: draw it, type it and pick one of five script fonts, or upload an image. Fill any date or text fields.
- 5 Click **I agree - sign this document**. The signed copy arrives by email once every signer is finished.

To get a working link without walking the admin UI, run `python api/tests/e2e/make_link.py` - it creates and sends an envelope, then prints the signer URL.

3. Working flow

Three stages. The first two are interactive; the third runs on a queue.

Stage 1 - the admin prepares and sends

Upload. The file's magic bytes are checked for the %PDF- header, then it is handed to the Python service, where a real parser confirms it opens, is not password-protected, and reports its page count. Only then is it stored in object storage under a random key - never the user's filename - and its SHA-256 recorded. That hash is the anchor everything downstream is compared against.

Field placement. pdf.js renders the pages and clicking drops a field for the selected recipient. Coordinates are stored as fractions of the page between 0 and 1 with a top-left origin, not pixels, so a placement survives zoom, screen DPI and the later conversion into PDF user space. Every field belongs to exactly one recipient, which is what makes "you cannot fill someone else's field" enforceable rather than merely hidden in the interface.

Send. Each recipient gets a 256-bit random token. Only its SHA-256 is stored, so a database dump yields no working links. Invitations are queued.

Stage 2 - the signer completes the ceremony

Signers have no account, so every request re-establishes who is calling from the token alone. The token is looked up by hash and compared in constant time; an expired link returns 410 and an unknown one 404.

Passcode. Six digits, hashed with bcrypt rather than SHA-256 - a million possibilities would fall to an offline sweep against a fast hash - expiring in ten minutes and locking out after five failures. The document is not served until this passes: possession of a forwarded link is not enough.

Consent. Records the disclosure version and a hash of the exact text shown, not a boolean. The question in a dispute is never whether someone consented but to what wording.

Signing. Typed names are rendered to PNG on the server, so the artefact sealed into the PDF does not depend on which fonts the signer happened to have installed. Uploaded images are fully decoded and re-encoded, which strips EXIF and anything else riding along with the pixels. Clicking the final button is logged as its own intent event, separate from having filled the fields in - that distinction is what the ESIGN Act and UETA actually turn on. The token is then burned.

Stage 3 - the server seals and delivers

Queued, because sealing makes a network round trip to a timestamp authority and fetches revocation data. That takes seconds and can fail for reasons nothing to do with the signer, so it retries with widening backoff rather than turning a transient outage into a failed signature.

The job reads field coordinates from the database, never from the finishing request, so a signer cannot move their own signature elsewhere in the document on the way out. It builds the certificate payload from the audit trail, then makes a single call to the Python service, which composites the marks, appends the evidence page and applies the seal in one pass - the half-finished document, carrying signatures but no cryptographic protection, never touches disk or crosses the network. The level actually achieved is stored, not the level requested.

Running underneath all of it

Every step writes a hash-chained audit event: each row's hash covers the previous row's, and PostgreSQL rejects UPDATE and DELETE on the table outright. That chain is printed into the Certificate of Completion, which then goes inside the sealed, timestamped PDF - so the record of what happened and the document itself end up cross-witnessing each other.

4. Question one: tech stack

Layer	Choice
Admin and signer UI	React 19, TypeScript, Vite 8, Tailwind 4, React Router 7
PDF rendering in the browser	pdf.js 6 (pdfjs-dist)
API	Laravel 13, PHP 8.3, Sanctum
Database	PostgreSQL 16
Queue and cache	Redis 7 (predis)
Object storage	S3, with MinIO in development
Email	SMTP - Mailpit locally, SES or Postmark in production
PAdES sealing service	Python 3.12, FastAPI, pyHanko 0.36
PDF composition	pypdf, reportlab, Pillow
Trusted timestamps	RFC 3161 - DigiCert
Certificates	OpenSSL dev CA publishing a CRL; a commercial CA or CCA-licensed ESP in production
Orchestration	Docker Compose
Tests	PHPUnit (API), Python integration scripts, end-to-end ceremony runner

5. Question two: why this stack, and why not the alternatives

Laravel for the API

The feature list is almost exactly Laravel's standard equipment: queues for the sealing pipeline, mail with an SES driver, an S3 storage abstraction, Sanctum tokens, authorisation policies, a scheduler for expiry sweeps. None of that had to be built. It is also the framework I have shipped production work in, so review effort went into the signing logic rather than into learning a framework.

React for the interface

Two screens carry real interaction state: the field-placement editor, with click-to-place, zoom and per-page coordinate mapping, and the signature pad, with canvas drawing, image upload and live font preview. A component model with local state fits that far better than server-rendered templates with sprinkled interactivity.

A Python sealing service - necessity, not preference

This is the most important decision in the project, and it was forced. **PAdES B-LTA cannot be produced in PHP with free libraries**. Four options were evaluated before adding a second language:

Option	Verdict
FPDI (free)	Cannot read PDF 1.5+ at all. PDF 1.5 introduced compressed cross-reference streams and object streams; the open-source parser does not support them, and most real-world PDFs are 1.5 or later. Reading them requires the commercial FPDI PDF-Parser add-on.
TCPDF setSignature()	Produces only a basic adbe.pkcs7.detached signature - no PAdES subfilter, no DSS dictionary, no document-timestamp chain. Structurally incapable of exceeding B-B.

Option	Verdict
SetaPDF-Signer	Implements PAdES properly. Commercially licensed.
pyHanko (Python)	Open source, covers B-B, B-T, B-LT and B-LTA with full long-term validation.

So the choice was: buy a licence, ship a weaker signature while describing it as something it is not, or add roughly 200 lines of Python behind one internal HTTP call. Only the last option delivers what was asked for, at no licence cost, with the cryptography handled by a library that specialises in it. The service is small and stateless - it holds no database credentials and takes bytes in, returning sealed bytes.

PostgreSQL rather than MySQL

The jsonb type with GIN indexing keeps audit payloads queryable without a second denormalised table, and CHECK constraints are expressive enough to enforce the envelope state machine and field-coordinate bounds in the database rather than only in application code.

One caveat found in practice: **jsonb does not preserve object key order**, returning keys sorted by length then bitwise. That broke the audit hash chain on the first run, because a payload written in one order re-serialised in another and the recomputed hash no longer matched. The fix was to canonicalise - recursively sorting keys - before hashing, the same idea as RFC 8785. The plain json type would preserve the text but lose the indexing.

PostgreSQL rather than MongoDB

The domain is inherently relational - envelope, recipient, field, value - and finalising a signature is a multi-table transaction. An evidence log needs strict ordering and referential integrity, which is the opposite of what eventual consistency offers.

Object storage rather than database blobs

PDFs run from 100 KB to 20 MB. Blobs bloat backups, slow replication and cannot be streamed. S3 also brings versioning, lifecycle rules, server-side encryption and pre-signed URLs at no additional effort.

Queues rather than synchronous sealing

Sealing involves a timestamp-authority round trip and revocation fetching. It must be retryable with backoff, not blocking an HTTP request.

Why not Next.js

A reasonable stack, but it would have meant learning a framework and implementing cryptography at the same time. The Python service is required regardless, so the single-language argument for a Node backend disappears.

Why not Node and Express

Queues, mail, storage and authentication would all have to be assembled by hand.

Why not sign in the browser with WebCrypto

The private key would have to reach the client. That is unacceptable key custody at any scale, and client-reported coordinates and timestamps cannot be trusted anyway. All sealing is server-side; the browser only ever produces a preview.

6. Question three: competition and cost

List prices verified August 2026. Annual billing unless stated; monthly billing typically carries a premium. Amounts in USD unless marked INR.

Global

Product	Price	Notes
DocuSign	Personal \$10/mo (\$15 monthly, 5 envelopes/mo); Standard \$25/user/mo (\$45 monthly); Business Pro \$40/user/mo (\$65 monthly); Enterprise custom	Annual plans capped near 100 envelopes per user per year. SMS delivery from \$0.40 per send, ID verification from \$2.50 per attempt
Adobe Acrobat Sign	Standard Individual \$12.99/user/mo; Pro Individual \$19.99; Standard Teams \$14.99; Pro Teams \$23.99; Acrobat Studio Teams \$29.99	Standard team capped near 150 transactions per user per year; monthly billing around 50 percent premium
Dropbox Sign	From about \$15/user/mo; free tier of 3 documents a month	API plans from about \$100/mo
PandaDoc	Essentials about \$19/seat/mo; Business about \$49/seat/mo	Document-generation focused
SignNow	From about \$8/user/mo	Cheapest mainstream option
Zoho Sign	Free tier; paid from about \$10/user/mo	

India

Structurally different - priced per transaction rather than per seat. That matters a great deal if signing is embedded in a product rather than used by a back-office team.

Product	Price
Leegality	Licence-free Basic tier: non-Aadhaar eSign about INR 15 per signatory, Aadhaar eSign about INR 25, e-stamping from INR 45 per paper
Digio	Contact sales; per transaction
SignDesk	Contact sales; per transaction
eMudhra emSigner / NSDL-Protean	CCA-licensed ESPs, per eSign transaction
Market range	INR 3 to INR 25 per signature, volume-dependent

Open source - the real alternative to building from scratch

Product	Price
DocuSeal (AGPLv3)	Self-host free; managed hosting from about EUR 9 per month per instance
Documenso (AGPLv3)	Self-host free; cloud from about \$30/mo; has PAdES support
OpenSign (AGPLv3)	Self-host free; cloud from about \$30/mo

Break-even

Ten administrators sending around 500 envelopes a month:

	Annual
DocuSign Business Pro, 10 seats	\$4,800 and above, before envelope overages
In-house running cost	About \$720 compute, \$60 storage, \$12 email and \$300 to \$500 for a document-signing certificate - roughly \$1,100

That looks like a clear win until the build is priced in: roughly 120 to 200 hours of engineering, plus ongoing patching, certificate renewal, timestamp-authority monitoring and backup verification. Realistic break-even is **twelve to twenty-four months**.

The honest conclusion: build in-house when signing is a feature of your product - embedded, white-labelled, per-transaction economics, deeply integrated with your own workflow and identity system. Buy when it is an internal back-office need. This project is worth building because it is the former; for a team that simply needs contracts signed, DocuSign is cheaper than the engineer-months.

7. Question four: security of building in-house versus buying

Real advantages of building it

- **Data residency and minimisation.** Contracts never leave your own network. Directly relevant to India's DPDP Act 2023 and GDPR Article 44 transfer rules: you choose the region, the vendor chooses theirs.
- **Blast radius.** An e-signature SaaS is a high-value aggregated target - one breach exposes every customer's contracts at once. A single-tenant instance is far less attractive and its compromise is not systemic.
- **Key custody.** The signing key lives in your own KMS or HSM. No third party can be compelled, phished or tricked into signing on your behalf.
- **Evidence ownership.** The audit trail is in your database in a format you control and can export. If a vendor account lapses or is terminated, their audit trails go with it - precisely when litigation makes you want them.
- **Access-control fidelity.** Signing permissions ride your existing roles and SSO. No per-seat pricing quietly encouraging shared accounts, which is a genuine security anti-pattern created by a commercial model.
- **No third-party code in the ceremony.** Vendor signing pages load analytics and CDN assets; a supply-chain compromise there sits directly inside the signing flow. This build serves its own fonts for that reason.
- **Deletion is real.** You can guarantee hard deletion and a defined retention policy. SaaS deletion is frequently soft-delete plus backups for several years.
- **No secondary use.** Your documents are not subject to a vendor's terms covering analytics or model training.

The counterweights, which matter just as much

- 1 You inherit patching, dependency vulnerabilities, certificate lifecycle, timestamp-authority availability, key rotation and backup-integrity testing. Permanently.
- 2 You have no SOC 2 Type II, ISO 27001, HIPAA BAA or 21 CFR Part 11 attestation. Enterprise buyers ask for these, and "we built it ourselves" is not an answer.
- 3 In a dispute, DocuSign's audit trail carries two decades of precedent and available expert witnesses. Yours is novel, and you must be prepared to prove your own process rather than point at an established one.
- 4 **In India, a legally recognised electronic signature under IT Act section 3A requires an eSign service from a CCA-licensed ESP bound to Aadhaar or PAN.** No amount of engineering substitutes for that licence.

The position this system takes

A hybrid. Own the interface, the storage, the workflow and the evidence; delegate identity binding and certificate issuance to a licensed authority. The code reflects this: a sealer seam lets the same envelope flow route either to the in-house PAdES sealer or to a CCA-licensed ESP such as Digio, Leegality or NSDL-Protean, without touching anything above it.

8. Question five: core concepts, and what a signature captures

An electronic signature is not a digital signature

An **electronic signature** is a legal concept: any electronic symbol or process attached to a record and executed with intent to sign. A typed name qualifies. A **digital signature** is a cryptographic mechanism binding a key to specific document bytes. The second is the technology commonly used to make the first trustworthy, but they are not the same thing, and conflating them is the most common mistake in this area.

Assurance tiers are jurisdiction-specific. Under eIDAS in the EU: simple, advanced, then qualified. Under India's IT Act 2000: section 3 covers a digital signature made with a certificate from a CCA-licensed authority, while section 3A covers electronic signatures listed in the Second Schedule, such as Aadhaar eSign. The US ESIGN Act and UETA are technology-neutral with no tiers at all - validity rests on intent, consent, attribution and record retention.

A consequence worth stating plainly: **in the US the evidence, not the cryptography, is what makes a signature enforceable**. That is why the audit trail in this system is treated as a first-class artefact rather than a log file.

What actually happens cryptographically

- 1 Define the byte range: every byte of the PDF except the gap reserved for the signature itself.
- 2 Hash that range with SHA-256.
- 3 Sign the digest with the signer's private key.
- 4 Wrap it in a CMS or PKCS#7 SignedData structure carrying the certificate chain and signed attributes.
- 5 Write it into the reserved gap as an incremental update, so every earlier revision of the document survives intact inside the same file.
- 6 A verifier recomputes the hash over the byte range and checks it against the signature. Any changed byte breaks it.
- 7 Trust comes separately, from the certificate chaining to a trusted root - Adobe AATL, the EU Trusted List, India's CCA or the operating system store - with revocation checked by OCSP or CRL.

A signature cannot sign itself: embedding it would change the bytes it just hashed. That is why the PDF reserves a hole and a ByteRange array declares which spans are covered.

Why timestamps and long-term validation matter

The signer's own clock proves nothing - it can be set to anything. An RFC 3161 timestamp authority countersigns the hash using a clock nobody in the transaction controls. Certificates also expire and are revoked, so PAdES defines escalating levels:

Level	Adds	Answers
B-B	PKCS#7 over the byte range	Has this been altered? Who held the key?
B-T	RFC 3161 timestamp	When was it signed, by an independent clock?
B-LT	Certificate chain and revocation data in a DSS dictionary	Still verifiable once the authority's endpoints are gone
B-LTA	A renewable chain of document timestamps	Still verifiable decades later, after the signing certificate expires

This system produces B-LTA, confirmed independently by pyHanko's command-line validator. A practical finding worth recording: B-LT and B-LTA require embeddable revocation data, and a bare self-signed certificate has no CRL distribution point, so it cannot reach those levels at all. With the CRL unreachable an otherwise perfect signature validates as INVALID; with it published, the same file validates cleanly. Publishing revocation data is not an optional extra.

What the ceremony captures - the evidence package

Category	Captured	Why it matters
Identity	Name, email, phone, IP - and how they were verified: link possession, email OTP, SMS, knowledge-based questions, Aadhaar or ID match	"Verified" without a method is not evidence of anything
Intent	An explicit affirmative act, recorded as its own event, distinct from filling fields in	UETA and ESIGN both turn on the signature being executed with intent to sign
Consent	Consent to transact electronically, with the disclosure version and a hash of the exact text, plus timestamp and IP	The question is never whether they consented, but to what wording
Attribution	IP, user agent, session, which token was used, every authentication event including failures	Failed attempts are evidence too
Timeline	Sent, delivered, opened, viewed, field completed, signed, completed - each in UTC	Establishes sequence, not just outcome
Document integrity	SHA-256 of the file as uploaded, after signatures are applied, and after sealing	Proves the starting point as well as the end state
The artefact	The signature image, whether drawn, uploaded or typed, the font if typed, and its page and coordinates	The mark itself is part of the record
Trusted time	An RFC 3161 token from an independent authority	Never the application server's clock
Tamper evidence	A hash-chained audit log and the PDF's own cryptographic seal	The record of events and the document are separately verifiable
Certificate of Completion	A human-readable PDF appended to the document summarising all of the above	In practice this is the page that actually gets read in a dispute

On the limits of a hash chain

Each audit event's hash covers the previous event's, so editing, deleting or reordering any event invalidates every hash that follows. An artisan command recomputes the whole chain and exits non-zero on a break, and PostgreSQL triggers reject UPDATE and DELETE outright.

It is worth being precise about what that achieves. A hash chain makes tampering **detectable**, not **impossible** - an attacker with write access could rebuild the chain from the point of the edit onward. What closes that gap is that the chain's contents end up inside a PAdES-signed PDF, timestamped by an authority outside your control. A rewritten chain can no longer be made to match the sealed document, and the timestamp cannot be backdated. Neither mechanism is sufficient alone; together they are strong.

9. Security controls implemented

- Signing tokens are 256-bit and stored only as SHA-256, so a database dump yields no working links. Tokens are burned on completion.
- A one-time passcode is required before the document is served; possession of a forwarded link is not enough. Passcodes are bcrypt-hashed and lock out after five failures.
- Every field is scoped to its recipient in the query itself, so another signer's field identifier resolves to nothing. Covered by an explicit insecure-direct-object-reference test.
- Uploaded images are fully decoded and re-encoded server-side, stripping EXIF and anything else riding along with the pixels. PDFs are validated by a real parser, not by file extension or declared content type.
- The audit table rejects UPDATE and DELETE at the database level, and each row's hash covers the previous row's.
- The sealing service has no public ingress and authenticates every request with a shared-secret HMAC over the exact request body.
- Login is throttled per email and IP pair, and answers identically for an unknown account and a wrong password so it cannot be used to enumerate users.

10. Verifying it works

End-to-end ceremony

```
./sign-service/.venv/Scripts/python.exe api/tests/e2e/ceremony.py
```

Drives the real HTTP API exactly as the SPA does: uploads, builds an envelope, sends it, reads the signing link and passcode out of Mailpit, signs, waits for the queue to seal, confirms the signed copy is delivered, then verifies the result and probes the access controls. **46 checks, all passing.**

The signature is real

```
cd sign-service
./venv/Scripts/pyhanko.exe sign validate --pretty-print \
--trust ../pki/out/ca.pem tmp/5-sealed.pdf
```

An independent validator, not this codebase, reports that the signature is cryptographically sound, that the timestamp is backed by a trusted authority (DigiCert SHA256 RSA4096 Timestamp Responder), and that the signature is judged **VALID**. In Adobe Acrobat Reader, after trusting the development CA certificate, the signature panel shows the document as signed with long-term validation enabled.

Tamper detection

```
cd sign-service
PKI_DIR=../pki/out ./venv/Scripts/python.exe scripts/smoke_test.py
```

Seals a document, then changes one byte of page content and re-validates. The altered file still opens perfectly and its signature reports as broken, which is exactly the point. The same file run through the pyHanko CLI returns **INVALID**.

Audit chain and test suites

```
cd api && php artisan signdesk:verify-audit # recomputes every chain
cd api && php artisan test # 27 tests
cd sign-service && ./venv/Scripts/python.exe scripts/http_test.py # 24 checks
```

The PHP tests run against PostgreSQL rather than SQLite. The schema depends on jsonb, GIN indexes and a plpgsql trigger, and testing against another engine would skip exactly the guarantees the tests exist to prove.

11. Known limitations

- **The development CA is not a public trust anchor.** Adobe shows the signature as valid only after its certificate is trusted manually. Production needs a certificate from a commercial authority, or a CCA-licensed ESP for recognition under IT Act section 3A. The sealer seam exists for exactly that swap.
- **Horizon is not used.** It requires the pctl and posix extensions, which are unavailable on Windows; the standard queue worker covers the same ground here.
- **No Vitest suite for the SPA.** The interactive behaviour that matters - canvas drawing and pdf.js rendering - needs heavy browser mocking to unit-test. Coverage came instead from the 46-check end-to-end run and a manual walkthrough of both portals. This is a real gap, not a design choice.
- **PDF rendering could not be confirmed in an automated browser.** pdf.js schedules rendering through requestAnimationFrame, which never fires in a browser pane that is not compositing, so the render promise cannot resolve there. Everything else in the signer flow was verified interactively.
- Single-signer routing is implemented and tested; multi-signer routing order is enforced server-side but has not been exercised end to end.

12. Sources

Pricing and library capabilities checked August 2026. DocuSign and Adobe Acrobat Sign pricing via Signeasy and Costbench. SignNow and Dropbox Sign comparison via Unkoa; Zoho Sign pricing from zoho.com. Leegality pricing via productgrowth.in; India eSign comparison via signyu.com and peko.one. Open-source alternatives via sliplane.io and everesign. pyHanko capabilities from docs.pyhanko.eu and the project repository. FPDF limitations from manuals.setasign.com. FreeTSA from freetsa.org.